

COSC 4P96 Project: PSO for Optimizing Topology Design of Neural Networks

GitHub: <https://github.com/eliasswitzer/cosc-4p96-project>

Kohen Serravalle
7462351
ks21xf@brocku.ca

Elias Switzer
7419997
es21dq@brocku.ca

Abstract—The increasing complexity of neural network design has driven the need for automated methods such as Neural Architecture Search (NAS). While many approaches have shown strong performance on established benchmarks, NAS remains relatively underexplored on the MedMNIST dataset suite. In this work, we investigate the effectiveness of Particle Swarm Optimization (PSO) as a search strategy for optimizing neural network architectures on MedMNIST datasets. PSO is a classic metaheuristic search algorithm and it is known for its simplicity and fast convergence, making it well-suited for NAS. Our approach optimizes full connected network topologies under a multi-objective framework balancing performance and model complexity. Results show that PSO approaches benchmark performance on single-label tasks but is less effective on multi-label problems. Larger neighborhood sizes and swarm populations were found to improve performance, while the use of a separate predictor model significantly reduced computational cost at the expense of some accuracy. Overall, PSO demonstrates effective balance of exploration and exploitation, identifies good architectures, and performs marginally better than random search.

I. INTRODUCTION

Designing effective neural network architectures often requires extensive manual tuning of hyperparameters and structural components, a process that is both time-consuming and computationally expensive. This challenge has motivated the development of automated approaches collectively known as Neural Architecture Search (NAS), which aim to systematically discover high-performing architectures with minimal human intervention.

The objective of this project is to investigate the use of Particle Swarm Optimization (PSO) as a search strategy for NAS. Specifically, we aim to leverage PSO to efficiently explore the architecture space and identify neural network configurations that achieve strong predictive performance while maintaining reasonable model complexity and training cost.

II. BACKGROUND AND RELATED WORK

A. Neural Architecture Search and Automated Machine Learning

Neural Architecture Search (NAS) is a subfield of Automated Machine Learning (AutoML), a broader area of research focused on automating the end-to-end process of applying machine learning to real-world problems. AutoML aims to reduce or eliminate the need for manual intervention

in tasks such as data preprocessing, feature selection, model selection, and hyperparameter tuning. Within this, NAS specifically addresses the challenge of automatically designing high-performing neural network architectures, replacing traditional trial-and-error approaches that are often time-consuming.

A typical NAS framework is composed of three key components. First, the *search space* defines the set of possible neural network architectures that can be explored, including choices such as number of layers and hyperparameters. Second, the *search strategy* determines how this space is navigated, with common approaches including reinforcement learning, evolutionary algorithms and Particle Swarm Optimization (PSO). Finally, the *performance estimation strategy* evaluates candidate architectures [1].

B. Particle Swarm Optimization

Particle Swarm Optimization (PSO) is a population-based metaheuristic optimization algorithm inspired by the flocking behavior of birds. In PSO, a set of candidate solutions, referred to as particles, move through a defined search space in order to identify optimal or near-optimal solutions through cooperative exploration.

Each particle has its own position, representing a candidate solution, and a velocity, which determines how it moves through the search space. The update of a particle's position is influenced by three components. First is the cognitive component which attracts the particle toward its own best-known position. Second is the social component which guides it toward the best solution found by the swarm (either global or local). Last is the inertia component, which preserves a fraction of its previous velocity to balance exploration and exploitation. Through iterative updates based on these components, the swarm collectively converges toward high-quality solutions.

The basic algorithm for PSO can be found in Algorithm 1.

C. Related Studies

A notable prior work in multi-objective NAS is *NSGA-Net*, which uses an evolutionary approach based on the Non-dominated Sorting Genetic Algorithm (NSGA). The search process is conducted in three stages: initialization using prior knowledge from hand-crafted architectures, exploration

Algorithm 1 PSO Algorithm

```
Set  $t = 0$ , initialize  $c_1, c_2, w$ 
Create initial swarm of particles randomly
Initialize velocities to 0
for Each particle do
    Evaluate fitness and set personal bests
end for
Determine global best particle (for tracking)
while Stopping conditions are not true do
    for Each particle do
        Calculate local bests
        Update velocity
        Update position
        Evaluate fitness
        Update personal bests
    end for
    Update global best (for tracking)
end while
```

through genetic operators such as crossover and mutation, and exploitation using a historical archive of previously evaluated architectures. This combination enables efficient traversal of the architecture space while maintaining a diverse set of high-quality solutions [2]. We will make use of some of these ideas, mainly initialization using prior knowledge.

Another approach is *Differential Architecture Search* (DARTS), which introduces a gradient-based method to address the performance and scalability limitations of traditional NAS techniques. Rather than training multiple architectures, it trains one neural network with all of the possible choices for each parameter that is being optimized. Each choice of a parameter has a weight that gets tuned via backpropagation. The network uses all of the possible choices by taking a weighted sum of each value for specific parameters. Despite its computational advantages, DARTS is known to be memory-intensive and can suffer from optimization instability [3].

Finally, *Population Historical Information-Driven Evolutionary Multitask NAS* (HIMT-NAS), extends evolutionary NAS to a multitask setting. Instead of optimizing architectures for a single task, HIMT-NAS searches for models that generalize well across multiple related tasks, leveraging shared knowledge to improve efficiency and performance. This approach addresses the high computational cost of single-task NAS and promotes knowledge transfer between tasks. Notably, it has been evaluated on medical imaging datasets, which aligns closely with the domain considered in this project [4]. The selected datasets will be explained further below.

III. PROPOSED APPROACH

A. Search Strategy

In this project, PSO is used to optimize the architecture of a fully connected multi-layer perceptron (MLP). PSO was selected due to its simplicity, ease of implementation, relatively small number of hyperparameters, and fast convergence characteristics.

We implemented the local-best (lbest) variant of PSO. In contrast to the global-best (gbest) variant, where each particle is influenced by the best solution found by the entire swarm, lbest PSO restricts this influence to a particle’s local neighborhood. This promotes diversity in the swarm and helps mitigate premature convergence. In our implementation, neighborhoods are defined using an index-based topology, controlled by a configurable neighborhood size parameter.

Several PSO hyperparameters are explored, including the inertia weight w , cognitive and social acceleration coefficients c_1 and c_2 , the number of particles, and the number of iterations. Additionally, velocity clamping is incorporated to prevent large updates that could destabilize the search process. The velocity bounds, v_{min} and v_{max} are defined as a proportion of each parameter’s search range.

The stopping criteria for the algorithm include reaching a maximum number of iterations, convergence of particle positions, or stagnation in the fitness values over some number of iterations.

B. Search Space and Particle Encoding

Each particle’s position is represented as a seven-dimensional vector, where each dimension corresponds to a specific architecture or training parameter of the neural network. The encoded parameters include hidden layer size, number of hidden layers, learning rate, momentum, batch size, weight decay, and dropout rate.

For example, a particle may look as follows:

$$[2 \quad 16 \quad 0.01 \quad 0.9 \quad 64 \quad 0.1 \quad 0.5]$$

This corresponds to an MLP with 2 hidden layers of size 16, a learning rate of 0.01, momentum of 0.9, batch size of 64, weight decay of 0.1 and dropout rate of 0.5.

Since some parameters are continuous and some are discrete, we round the values of the discrete parameters to the nearest whole number after a position update and before fitness evaluation.

The number of training epochs is not included in the particle encoding. Incorporating it as an optimization variable would make it difficult to fairly compare architectures, as models trained for more epochs may achieve better performance simply due to longer training. Instead, the number of epochs is held constant across all evaluations to ensure a consistent and unbiased comparison of candidate architectures.

C. Multi-Objective Optimization

This project formulates NAS as a multi-objective optimization problem, balancing model performance against model size and complexity. The goal is to identify architectures that achieve high predictive performance while remaining computationally efficient.

The performance objective depends on the task setting. For single-label, multi-class classification, performance is measured using accuracy, which is maximized. For multi-label classification, several metrics were considered and Hamming

loss was ultimately selected due to its ability to capture per-label prediction errors. In this case we want to minimize Hamming loss. To ensure consistency within the framework, all objectives are reformulated as maximization problems. Specifically, Hamming loss is transformed by using $1 - \text{loss}$.

Model complexity is incorporated as a secondary objective, measured through the ratio of the number of parameters in the model to the maximum possible number of parameters for a model within our search bounds. This is a good measurement of complexity because it normalizes the complexity values to make the aggregation more stable. It is also transformed into a maximization problem by taking $1 - \frac{\text{number of parameters}}{\text{max number of parameters}}$, so it is a simplicity score of sorts.

To combine these objectives, we use a weighted sum fitness function that balances performance and complexity according to predefined coefficients α and β .

The decision to use a maximization framework was primarily a design choice for consistency. Since performance metrics such as accuracy are naturally expressed as maximization objectives, the remaining components were transformed accordingly. An equivalent framework using minimization could have been used, however a single optimization direction was selected for consistency.

D. Constraint Handling

Our implementation enforces feasibility through predefined search bounds for each parameter in the architecture space. For example, the number of hidden layers is restricted to a range of 1 to 5. These bounds ensure that all candidate solutions remain valid and practical. In some cases, the constraints are inherent to the parameter itself, such as dropout rate needing to be within a range of 0 to 1, while in other parameters they are imposed to limit model size and prevent excessive computational cost.

Constraint handling is performed using a combination of boundary correction and a penalty function. First, if a particle's position exceeds the defined bounds, it is brought back into the feasible region by clipping each dimension to its corresponding minimum or maximum value. This guarantees that all evaluated solutions satisfy the problem constraints.

In addition, a penalty function is applied to discourage particles from repeatedly going outside the search bounds. The penalty reduces a particle's fitness proportionally to the magnitude of its constraint violation prior to clipping. This mechanism guides the swarm away from infeasible regions of the search space while still allowing exploration near the boundaries.

E. Performance Estimation Methods

We evaluated two approaches for estimating the performance of candidate architectures during the search process. The first involves directly training each candidate model for a fixed, small number of epochs and using the resulting validation metrics as an estimate of its performance. While this method provides relatively accurate evaluations, it is

computationally expensive due to the need to repeatedly train models throughout the search.

The second approach uses a separate model trained on data collected from previously evaluated architectures, referred to as the performance predictor. This lightweight model estimates the performance of new candidate architectures without requiring full training, significantly reducing computational cost. However, this comes at the expense of reduced accuracy in performance estimation, which may impact the overall quality of the search.

This trade-off will be discussed further in the analysis section.

F. Elite Initialization Ratio

Elite initialization is a particle configuration method that initializes particles by picking from a list of pre-evaluated 'elite' particles. These elite particles were obtained from previous runs that already have good fitness, effectively giving the particle a head start. This is important as random initialization may add increased running time if particles start in bad regions. Elite initialization avoids this issue since elite particles will already start in good regions of the search space. Despite this, random initialization is still beneficial as it provides a mechanism for generating diverse particles. This leads to the notion of the elite initialization ratio or probability. It is the chance for a particle to be initialized either by elite initialization or random initialization and is a value between 0 and 1. Elite initialization ratio seeks to leverage the beneficial aspects of both initialization strategies, that is, the diversity from random initialization and the head start provided by elite initialization.

IV. EXPERIMENTAL SETUP

A. Datasets

The datasets used in this study are drawn from the MedMNIST benchmark suite, which provides a standardized collection of lightweight medical imaging datasets. Leveraging a benchmark dataset enables direct comparison with existing methods. MedMNIST consists of 28 x 28 images from various medical domains, including chest x-rays, retinal scans, and blood cell microscopy, and it supports a range of classification tasks [5].

The specific tasks vary across sub-datasets. For BloodMNIST, OCTMNIST, and TissueMNIST, the objective is standard multi-class classification, where each image is assigned to exactly one class. In contrast, ChestMNIST presents a multi-label classification task, where each image may belong to zero, one, or multiple classes simultaneously. Our implementation supports both settings, allowing us to evaluate how effectively the proposed approach generalizes across different problem types.

For consistency and comparability across experiments, the majority of the results presented in this work are based on BloodMNIST.

Figures 1 and 2 show sample images from BloodMNIST and ChestMNIST.



Fig. 1: Sample image from BloodMNIST

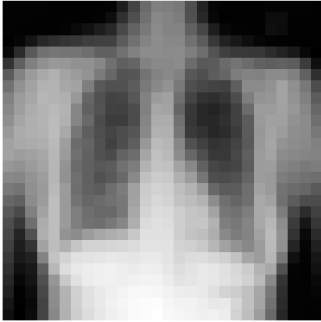


Fig. 2: Sample image from ChestMNIST

We use the official training, validation and testing splits provided by MedMNIST. The training and validation sets are used during particle evaluation to train candidate architectures and estimate their performance. The test set is reserved for evaluating the final best architecture identified by the search process, ensuring an unbiased assessment of generalization performance.

Across the MedMNIST datasets we use, the data is typically partitioned into approximately 70% training, 10% validation, and 20% testing, but this can vary.

B. Baseline for Comparison

The primary baseline that will be used for comparison is random search. It provides a meaningful point of reference to evaluate whether our more sophisticated approach offers significant improvements in efficiency and solution quality. To ensure that observed differences in performance are not due to random variation, we will conduct statistical significance testing on the results.

Random search is the process of randomly initializing n particles within search bounds and selecting the particle with the highest fitness. It can be simulated in the PSO framework by taking the largest initial fitness value out of a population of n particles.

C. Parameter Configurations

The parameter configurations used for the main experiments were set as default in the implementation. These include $w = 0.729$, $c_1 = c_2 = 1.49445$, neighborhood size of 3,

30 particles, 20 PSO iterations (usually stops earlier) and each model is trained for 10 epochs. To get more robust results, metrics from three runs were averaged. These runs used random seeds 10, 20, 30. For ChestMNIST specifically, due to its multi-label nature and it taking longer to train, we only trained each model for 5 epochs and it is only run once with seed 30. Visualizations from some of these runs will be found in the results section.

The number of particles and neighborhood size are experimented with as well, trying larger and smaller numbers for both. These experiments were only run using one seed (30) and were run using the predictor.

To compare PSO to random search, random search was run with the exact same setup except for the aforementioned changes to w , c_1 and c_2 .

For testing elite initialization, experiments were done with a 0%, 25%, 50%, 75% and 100% elite initialization ratio using the BloodMNIST dataset. The performance predictor was used, with a population of 15 particles. The fitnesses were averaged over 3 runs.

Additionally the search space bounds can be found in Table I.

Parameter	Min	Max
Hidden Layers	1	5
Hidden Layer Size	16	512
Learning Rate	0.0001	0.01
Momentum	0	0.99
Batch Size	16	256
Weight Decay	0	0.001
Dropout Rate	0	0.5

TABLE I: Experiment search bounds

V. RESULTS

A. BloodMNIST and ChestMNIST Results

The following tables (Tables II and III) showcase the accuracy and AUC of our implementation on both types of tasks compared against the highest benchmark for the same dataset. ChestMNIST values are reported from a single run and should be interpreted cautiously.

Dataset	Accuracy (Ours)	Accuracy (Benchmark)
BloodMNIST	0.882 ± 0.0037	0.966

TABLE II: Accuracy comparison of best found model

Dataset	AUC (Ours)	AUC (Benchmark)
BloodMNIST	0.9862 ± 0.0009	0.998
ChestMNIST	0.6596	0.778

TABLE III: AUC comparison of best found model

B. Search Behavior Results

1) *Single-Label Behavior*: This section shows the search behavior of one of the BloodMNIST dataset runs, including best/average fitness (Figure 3), overall swarm diversity (Figure 4), architectural distributions (Figure 5), and a Pareto front (Figure 6).

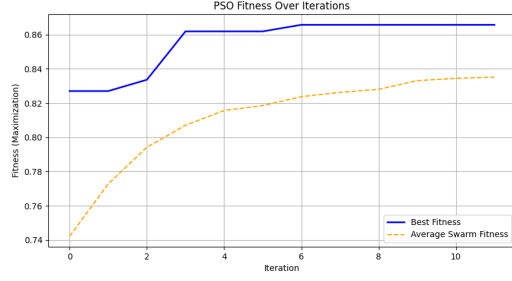


Fig. 3: BloodMNIST Best and Average Fitness by Iteration

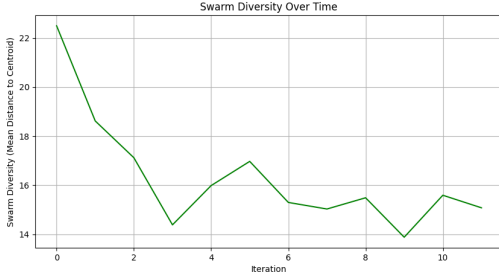


Fig. 4: BloodMNIST Diversity by Iteration

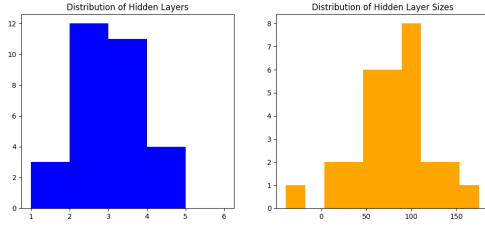


Fig. 5: BloodMNIST Hidden Layer Distribution

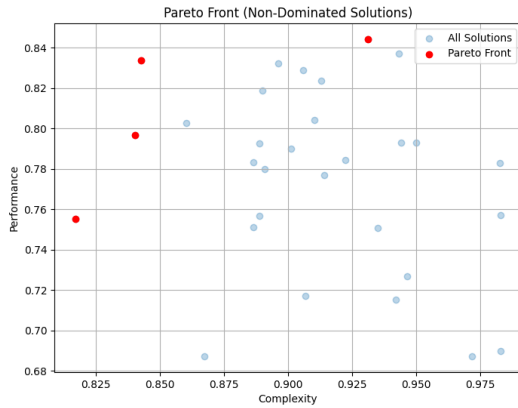


Fig. 6: BloodMNIST Pareto Front

2) *Multi-Label Behavior*: This section shows the search behavior of one of the ChestMNIST runs in Figures 7, 8 and 9.

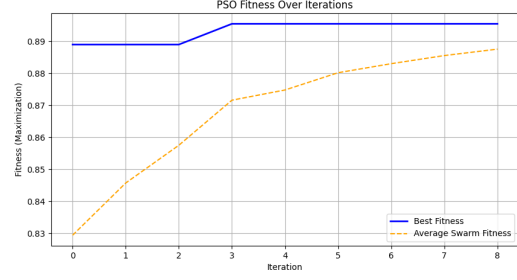


Fig. 7: ChestMNIST Best and Average Fitness by Iteration

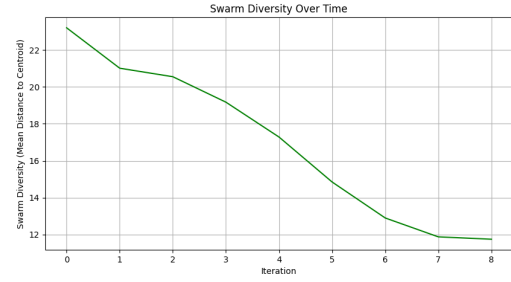


Fig. 8: ChestMNIST Diversity by Iteration

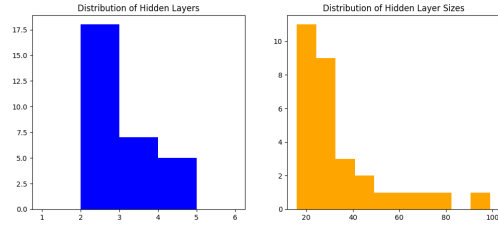


Fig. 9: ChestMNIST Hidden Layer Distribution

C. Design Choice Results

Figures 10 and 11 show the effect of changing the neighborhood size and number of particles in the PSO algorithm respectively.

D. Elite Initialization Results

Table IV shows the average fitness of the model at different elite initialization thresholds.

Elite Int. Value	Avg. Fitness
0%	0.83987
25%	0.84060
50%	0.84177
75%	0.83823
100%	0.83640

TABLE IV: Fitness results of varying elite initialization values

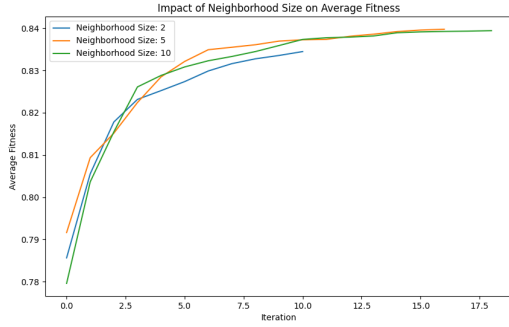


Fig. 10: Effect of neighborhood size on average fitness

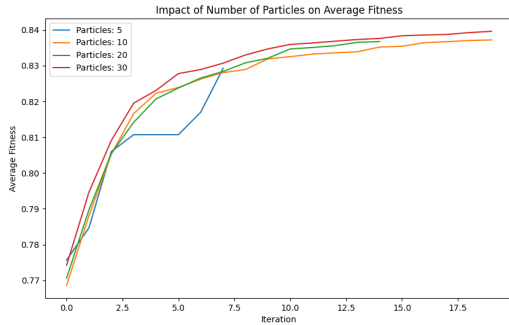


Fig. 11: Effect of number of particles on average fitness

VI. ANALYSIS AND DISCUSSION

A. Comparison to MedMNIST Benchmarks

Based on the achieved accuracy and AUC of the best-performing architectures, our method approaches the benchmark performance on BloodMNIST, but there is still room for improvement for ChestMNIST. This outcome is expected, as ChestMNIST represents a more challenging multi-label classification task. Additionally, our optimization objective for multi-label data was not accuracy, so we don't have that direct comparison.

It is also important to note that the benchmark results reported in MedMNIST are typically achieved using highly optimized architectures, such as ResNet, whereas our approach is limited to fully connected multi-layer perceptrons. Despite this architectural limitation, the results demonstrate that MLPs can achieve competitive performance on simpler datasets like BloodMNIST when appropriately tuned.

Overall, these results highlight the effectiveness of PSO in optimizing neural network configurations and the limitations of MLPs compared to more advanced architectures.

B. Swarm Behavior and Architecture Distribution

For both BloodMNIST and ChestMNIST, the swarm shows a steady improvement in average fitness over iterations, as indicated by the smooth upward trend that gradually plateaus in Figures 3 and 7. This behavior suggests that the PSO algorithm is effectively balancing exploration and exploitation, allowing

the swarm to iteratively refine candidate architectures. The occasional discovery of new global best solutions further demonstrates that the search process continues to explore promising regions of the search space.

In addition, the diversity of the swarm decreases over time, as seen in Figures 4 and 8, indicating convergence of particle positions. This reduction in diversity occurs gradually rather than abruptly, suggesting that the algorithm avoids premature convergence while still achieving stable optimization.

Analysis of the resulting architecture distributions provides further insight into the search dynamics. For both datasets, architectures with 2 hidden layers are most common, followed by 3 layers, suggesting that moderately deep networks provide a good balance between expressiveness and complexity for our datasets. Differences in most common hidden layer sizes across datasets are also observed. For BloodMNIST, larger hidden layer sizes (approximately 100 units) are more frequent. In contrast, ChestMNIST has significantly smaller hidden layers (around 20 units). One possible explanation for this is increased difficulty and multi-label nature of the task.

C. Impact of PSO Parameter Choices

The impact of neighborhood size is evident when examining the average fitness achieved across different configurations. Smaller neighborhood sizes tend to reach stagnation earlier, resulting in premature termination of the search process. This behavior can be attributed to limited information sharing, where particles rely on a smaller subset of neighbors and may converge more quickly to local optima. In contrast, larger neighborhood sizes promote greater diversity by allowing information to propagate more broadly throughout the swarm, supporting continued exploration. However, beyond a certain point we see the benefits diminish, as shown by similar performance observed for neighborhood sizes of 5 and 10. This suggests that there is a moderate window of neighborhood sizes that provide a good balance between exploration and convergence.

The number of particles also has a significant effect on the performance of the PSO algorithm. Increasing the swarm size enhances diversity and improves coverage of the search space, which can lead to better solutions. However, this comes at the cost of increased computational time due to the need to evaluate more candidate architectures per iteration. In Figure 11, we see that the best average fitness is achieved with 30 particles, but configurations with 20 and even 10 also yield comparable results. This suggests that smaller swarms may offer a better trade-off between performance and computational cost for these tasks.

D. Impact of Elite Initialization

In table 12, we can see a slight increase in fitness when the elite initialization ratio is around 25% and 50%, which is what was expected. 50% being the highest fitness demonstrates that the idea of an initialization strategy utilizing both elite and random initialization has some merit. We also observe a fitness decline when the elite initialization ratio is 75% and 100%.

This reinforces the idea that the lack of particle diversity when using close to a pure elite initialization can be detrimental to fitness.

A t-test was performed on the two groups with the biggest mean difference, that being 50% and 100%. The result concluded the two groups are not statistically different with a p-value of 0.1025. Given the small sample size, more experiments should be performed in the future to determine if this holds true for a much larger sample size.

E. Performance vs Cost Trade-Offs

Examining the Pareto front for BloodMNIST in Figure 6, we observe that the proposed method is capable of identifying several high quality non-dominated solutions. Although our PSO approach relies on a weighted aggregation fitness function rather than an explicit Pareto-based optimization strategy such as NSGA-II, we analyze the final swarm by plotting all particles and identifying those that are non-dominated. These solutions, highlighted in red, represent the best trade-offs between model performance and complexity, with four such points identified in the final iteration.

This analysis demonstrates that, despite not explicitly optimizing for a Pareto front, PSO is still able to discover a diverse set of competitive solutions that balance performance and cost effectively. Overall, the results indicate that the method achieves a strong trade-off, providing multiple viable architecture choices depending on the desired balance between accuracy and model size.

F. Performance Predictor Analysis

The performance predictor is a way to predict particle accuracy quickly, without having to train an entire model on the particle's architecture to determine its accuracy. It is a regression model trained on the particles themselves, returning how it thinks the particle would have performed. The training data consists of the particle's encoded architecture as input for the model, with labels being the accuracy of a neural network trained using the particle's parameters.

1) *Initial Architecture and Performance of the Performance Predictor:* Because this data is expensive and has to be generated manually, it was imperative to ensure that the model would generalize despite the lack of data. Overfitting was mitigated in the performance predictor by minimizing model complexity. The number of epochs was empirically determined to be around 300 epochs. The amount of training data also played a pivotal role in model performance. 700 pieces of training data were found to generalize the best, while numbers up to 1000 actually increased generalization loss.

The initial performance predictor model had the following losses:

Avg. Training Loss	Avg. Generalization Loss
0.0004197	0.00082724

These two losses being similar indicates the model is not overfitting much and is generalizing quite well. Even with the validation loss being as low as it is, the particle predictor was

still imperfect. This prompted further action to improve the particle predictor by primarily modifying the training data and its loss function.

2) *Improving the Initial Performance Predictor:* Several ideas were proposed to improve this baseline version of the performance predictor with hopes to decrease generalization loss further and get more accurate predictions. Our first idea focused on addressing the imbalance of the training data. Most architectures in the training data were predominately in the upper echelons of 80% and 70% accuracy, meaning the performance predictor classifies models that fall in these regions more accurately. To get a better view of the dataset imbalance, this bar chart depicts the frequency of the training data to fall in a certain decade of percentages.

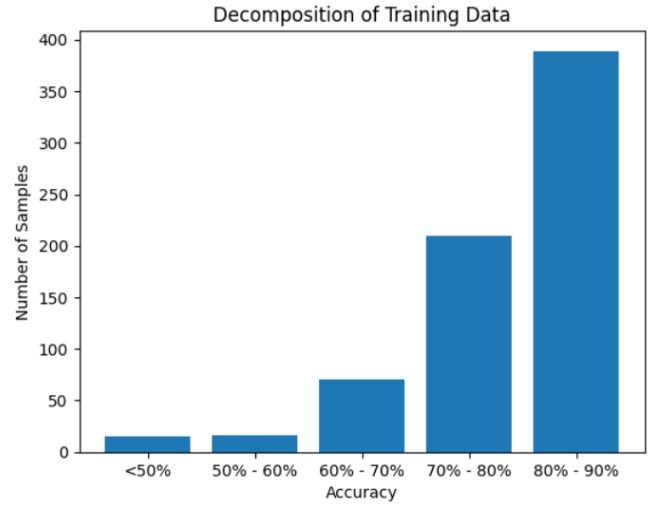


Fig. 12: Frequency of training data particle accuracy

The first idea addressed the dataset imbalance by discarding 50% of the training data in the highest echelon to encourage the model to learn the patterns of the other categories better. This idea had mixed success; the model could not perform as well as it could without a significant portion of its training data discarded. The second idea leveraged the fact that the accuracy of the model for the lower echelons of accuracy is an irrelevance; as long as the model can determine that an architecture is suboptimal, that would be sufficient. Before training data was fed to the predictor, if it was below a certain threshold of accuracy, it would be set to a value such as 0.5 to show that this model has inadequate accuracy. The rationale behind this is to prevent the model from wasting resources determining if an already poor performing model is worse than another poor performing model, and would instead focus on differentiating between better performing models more. The final idea saw the creation of a custom weighted MSE loss function to better penalize labels in lower echelons. This approach aims to encourage the model to equally prioritize learning by increasing loss for more and more infrequent echelons.

3) *Analyzing the Proposed Improvements of the Performance Predictor:* The following table shows the average

generalization loss of all the proposed ideas mentioned in the previous section. Additionally, statistical tests were performed with each idea against the baseline to test if the changes result in a statistically significant difference in the average generalization accuracy.

Remedy	Avg. Generalization Acc.	P-value
Baseline	0.000827243747538887	—
First Idea	0.002401545068481937	0.0018
Second Idea	0.0010155957324896009	0.1253
Third Idea	0.003211867953184992	0.0005

The statistical tests conclude that the first and third ideas result in an average generalization accuracy that is statistically outperformed by the baseline model, with the second idea being statistically similar to the baseline model. This suggests that the idea of focusing the model’s efforts to learning patterns of good performing models has some merit and may be worth exploring further.

Because none of the proposed ideas were found to statistically outperform the baseline model, the performance predictor was chosen to be implemented using the baseline model. After training enough models using the baseline, one was eventually found that was considerably lower in generalization accuracy. This model generalized the best out of all models trained and was therefore the one selected to be used as the final performance predictor.

4) *Comparing PSO Results with the Performance Predictor*: In this section, we will see how the quality of solutions changes from using the performance predictor. The following two tables show the complexity score and accuracy of particles trained traditionally vs predicted using the performance predictor. The particles were trained to 10 epochs, and each method had a population of 5 particles.

Iteration	Performance (Acc.)	Complexity Score
1	0.8232	0.8951
2	0.7940	0.9313
3	0.8319	0.8440
4	0.8101	0.8716

TABLE V: Performance and Complexity Score for training particles

Iteration	Performance	Complexity Score
1	0.7677	0.9820
2	0.7777	0.9682
3	0.7923	0.9629
4	0.7732	0.9821

TABLE VI: Performance and Complexity Score for predicting particle performance

The results show that taking the time to fully train the particles results in better accuracy than predicting them using the particle predictor. A t-test was performed on the performance values in both tables. A p-value of 0.0091 was recorded, meaning the results of fully training particles outperforms the results predicted by the performance predictor. This was expected, as the performance predictor would introduce a

small amount of error for its predictions. The fact that the highest particle predictions by the particle predictor also had substantially high complexity scores allude that complexity plays a significant part in how the performance predictor evaluates particles.

We will now compare the fitness values from particles predicted by the performance predictor and particles trained traditionally, as shown in the following table. The fitness is the result of aggregating the metrics into a single value. To prioritize accuracy over complexity, the performance weight (α) was set to 0.7 and the complexity score weight (β) was set to 0.3. The data demonstrates how both methods of

Iteration	Predicted Fitness	Trained Fitness
1	0.8320	0.8448
2	0.8348	0.8352
3	0.8439	0.8355
4	0.8359	0.8286

TABLE VII: Fitness for predicting particle performance

evaluating particles result in very similar fitness values. Using the performance predictor yields particles lower in accuracy but with a higher complexity score. Opting to fully train particles instead leads to the opposite, yet both methods of particle evaluation result in similar fitness values.

Overall, this section demonstrated an important trade-off between predicting particle performance vs determining particle performance through training. While using the performance predictor is significantly quicker, it may result in decreased confidence in particle accuracy. Training a particle gives better confidence in its accuracy value, but it does take significantly longer. When both metrics are combined in our multi-objective formula, the results from either evaluation method are found to yield similar fitness values.

G. Random Search vs Our PSO Method

In this section the PSO algorithm is compared to a random search approach of NAS, where n particles are randomly generated and evaluated, and the one with the highest fitness is selected. A statistical test using average fitness was performed with a population size of 5 for the PSO algorithm and $n = 5$ for the random search. 10 samples were taken to ensure a more accurate reading of statistical significance. The following table shows the result of a t-test on the PSO algorithm and random search.

Search Strategy	Avg. Fitness	P-value
PSO	0.83364	0.0472
Random Search	0.81235	0.0472

TABLE VIII: PSO vs Random Search: Avg. Fitness and P-value

With a standard α value of 0.05, the t-test results show that PSO results are statistically significant by a small margin.

The table of the 10 runs of random search and PSO provide a more comprehensive view of the differences between the two search strategies. PSO consistently gets high fitness values

Iteration	Random Search Fit.	PSO fit.
1	0.8261	0.8327
2	0.8321	0.8372
3	0.8298	0.8347
4	0.7816	0.8408
5	0.8123	0.8361
6	0.8171	0.8394
7	0.8193	0.8041
8	0.8337	0.8339
9	0.8315	0.8380
10	0.7400	0.8395

TABLE IX: Fitness results from 10 runs of random search and PSO

while random search results are slightly lower and have more variance. Overall, this demonstrates that using PSO for NAS can achieve high quality results consistently, at the cost of increased computational complexity. However, if finding the best possible solution is not a requirement, then random search becomes more viable as the extra complexity from PSO becomes unnecessary.

VII. CONCLUSION

The results of PSO demonstrate a clear effectiveness for NAS, approaching benchmark results for single-label datasets. PSO as a search strategy performed well, consistently achieving high fitness values. The proposed elite initialization ratio was shown to be statistically similar, but further experiments involving larger sample sizes are needed to verify this conclusion. Testing results with the performance predictor determined the baseline predictor to be statistically better or similar to any proposed improvements, prompting the selection and optimization of the original performance predictor. The performance predictor was found to provide extremely fast computation at the cost of full confidence of produced accuracy values. To conclude, while PSO could reliably outperform a search method like random search, the difference is marginal and incurs an extra computational cost, demonstrating a trade-off between PSO and random search.

VIII. FUTURE WORK

Several options for future work could further enhance this project. First, the approach could be scaled to larger and more complex medical imaging datasets to better evaluate its generalization and robustness. In addition, further improvements to computational efficiency could be explored, particularly through more advanced performance prediction methods.

Hybrid optimization methods also could present an improvement. For example, combining PSO with other metaheuristic methods such as Ant Colony Optimization (ACO) may improve exploration and convergence.

From an architecture perspective, the search space could be expanded to include additional architectural features for MLPs, or extended to more expressive models such as convolutional neural networks (CNNs). While this would likely improve performance, it would also introduce increased computational cost (so the improvements in computational efficiency would be vital).

Finally, future work could involve developing alternative predictors models that estimate metrics such as F1-score rather than accuracy.

REFERENCES

- [1] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.
- [2] Zhichao Lu, Ian Whalen, Vishnu Boddeti, Yashesh Dhebar, Kalyanmoy Deb, Erik Goodman, and Wolfgang Banzhaf. Nsga-net: neural architecture search using multi-objective genetic algorithm. In *Proceedings of the genetic and evolutionary computation conference*, pages 419–427, 2019.
- [3] Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018.
- [4] Kunjie Yu, Hao Tang, Jing Liang, Chao Li, and Mingyuan Yu. Population historical information-driven evolutionary multi-task neural architecture search. *IEEE Transactions on Neural Networks and Learning Systems*, pages 1–15, 2025.
- [5] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. MedMNIST v2 - a large-scale lightweight benchmark for 2d and 3d biomedical image classification. *Scientific Data*, 10(43), 2023.